LAB 1 REPORT · KAIROS DATASET

Did the post beat its author’s usual level?

Predicting LinkedIn post engagement relative to the author’s own history, from metadata, author history and post text.

Pavel Rodygin

Social Media Analytics, Agents & LLMs · ITMO University · October 2026

0.594	0.541	+0.053
macro-F1, final model	macro-F1, official baseline	absolute gain

Three ordered classes, defined against each author’s own past

The label compares a post with its author’s usual reach, so raw popularity does not give the answer away:

```
level = median log(1 + reactions)
        over the author's earlier posts (≥ 5)
resid = log(1 + reactions) − level
```

Cutoffs are the 20th / 80th percentiles of the train residuals and are applied unchanged to test. An author’s first five posts and plain reposts carry no label.

CLASS	REACTIONS VS USUAL	TEST SHARE
0 · below	under ×0.577	20.1%
1 · typical	in between	59.5%
2 · above	over ×3.182	20.4%

Metrics

- **macro-F1** (headline): weighs the classes equally, so “always typical” scores only 0.25.
- **QWK**: the classes are ordered, so calling a *below* post *above* is penalised more than calling it *typical*.

2 · DATA

5,433 LinkedIn authors, split by author

151,351
posts in the feeds, reposts included

89,018
labelled own posts

161,589
media files (images, PDFs, video), not used here

Split: 80 / 20 by author, seed 42

- **Train:** 3,295 authors, 71,395 posts
- **Test:** 824 authors, 17,623 posts
- All posts of one author sit on one side, so the model has to generalise to *unseen authors*.

What I used

- Post text (pseudonymised: `⟨URL⟩`, `⟨PERSON_n⟩`, …), content type, timestamps, follower counts.
- Only the text part of the 50.5 GB core archive was extracted (posts, authors, labels, split, 0.5 GB). Images and videos are left for future work.

Three feature blocks fused into one gradient-boosting model

baseline

Post & author metadata

Length, words, lines, hashtags, mentions, links, question mark, content type, hour, weekday; author level, IQR, followers, days since previous post; post age at crawl.

+

new

Extended author history

Recent momentum and spread computed only from **earlier** posts: last 1 / 3 / 5 posts vs level, std, max / min vs level, trend, posts in the last 30 days, relative length.

+

new

Text model (stacked)

Word 1–2-gram + char 3–5-gram TF-IDF → class-balanced logistic regression → 3 class probabilities, out-of-fold on train.

→

HistGradientBoosting

500 iterations, lr 0.05, 31 leaves, L2 = 1.0, **class_weight = balanced**. Output: below / typical / above.

Why these choices

- The label is *relative*, so what helps is deviation from the author's habits, not absolute popularity.
- Balanced class weights stop the model from defaulting to *typical*, which macro-F1 punishes.

Fusion design

- Stacking (late fusion): the text model runs separately and passes its probabilities to the booster as features, so sparse text and dense metadata never have to share a feature space.

How I worked: a decision log, in the order things happened

1 Read the label first

The target is relative to each author. Absolute popularity (followers, raw reactions) says *who* wrote a post, not whether it beat that person's norm, so I looked for features describing **deviation from the author's habits**.

2 Fix a reference point

Reproduced the official baseline (0.541) and re-ran its feature set through my own booster (0.539), so every later gain is measured against the same setup.

3 Accept a data constraint

The core archive is 50.5 GB and time was short. I extracted only the text part (posts, authors, labels, split, about 0.5 GB) and consciously left images and video out of scope.

4 Test a hypothesis: streaks

If an author's recent posts beat their long-run median, the next one may too. I built momentum and spread features from **earlier posts only**. Result: +0.033 macro-F1.

5 Secure a result, then go further

Ran the fast no-text variant first (about 15 s) to lock in a valid submission before the slower text model (about 7 min).

6 Add text the cheap way

TF-IDF + logistic regression, stacked as out-of-fold probabilities. Alone it is weak (0.452), but it is complementary to history: +0.022. I stopped there and listed what to try next.

5 · DECISIONS

Each choice, the alternative I rejected, and why

WHAT I CHOSE	ALTERNATIVE	WHY
Features of deviation from the author's norm	Raw reaction counts, follower count as the main signal	The label is relative, so a model on raw counts learns who wrote the post rather than how the post did.
Text as stacked LR probabilities	Concatenate ~400k sparse TF-IDF columns into the booster	Trees handle dense features better, and a linear model is a strong, fast text learner. Stacking keeps the two separate.
Out-of-fold, `GroupKFold` by author	Plain K-fold, or in-sample probabilities	An author's style would leak across folds. Test authors are unseen, so validation has to match.
Balanced class weights	Unweighted model + argmax	With a 20 / 60 / 20 split it drifts to *typical*, and macro-F1 punishes exactly that.
No normalisation, char n-grams	Lower-casing, stemming, stripping emoji and placeholders	Emoji, hashtags and ⟨PERSON⟩ tags carry signal, and char n-grams survive informal spelling (Tutorial Q1).
Gradient boosting on tabular features	Deep network or per-post LLM calls	Mixed tabular data, a 15-second training loop, readable feature effects. Dataset rules also keep media out of external services.
Skip media for now	CLIP embeddings of images and PDF covers	50.5 GB core + 114 GB video under a time limit. Securing text + metadata first gives a measured base to compare against.

No feature sees the outcome it is asked to predict

History features: shift before aggregating

Each author's posts are sorted by time, and every rolling statistic starts from `shift(1)`, so a post never sees its own reactions, as in the label definition.

FEATURE	MEANING
`last3_rel`	mean of last 3 posts − level
`trend5`	last 5 vs the 5 before them
`hist_std`	spread of the author's past
`posts_30d`	posting frequency
`text_len_vs_author`	length vs author's usual

Text probabilities: out-of-fold, grouped by author

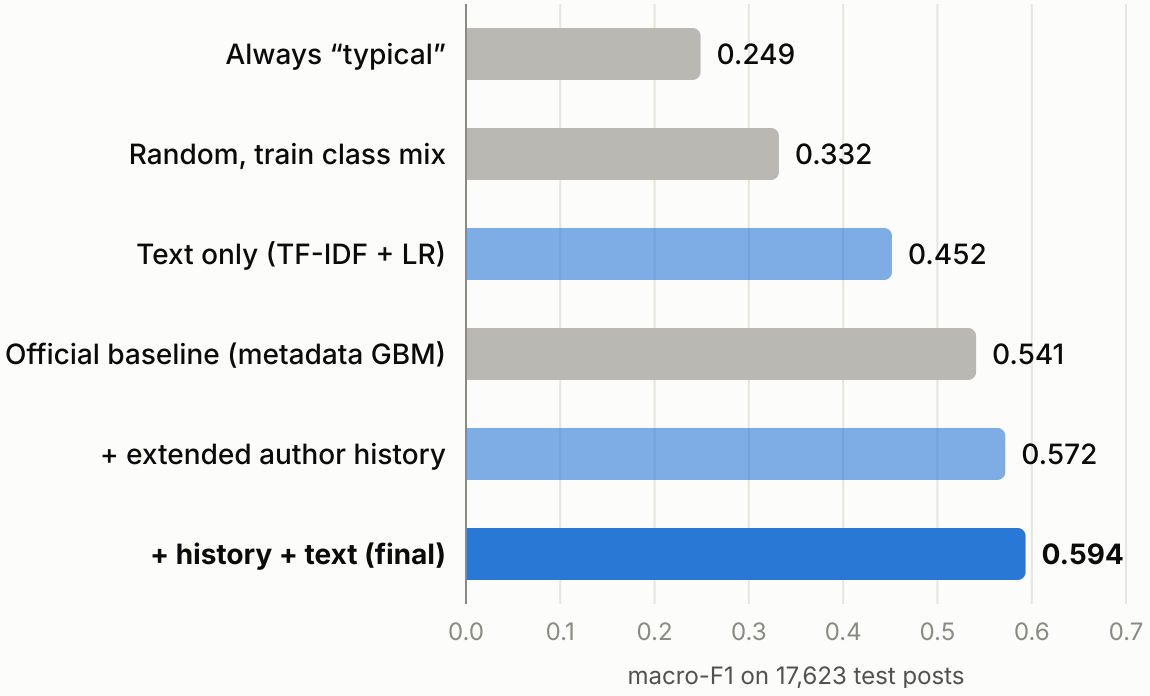
Fitting TF-IDF + LR on all of train and feeding its train-set predictions to the booster would leak: the booster would trust over-confident in-sample probabilities.

- 5-fold `GroupKFold` with author as the group: each train post gets probabilities from a model that never saw its author.
- Test probabilities come from a model refit on all of train.
- This mirrors the test setting, where authors are unseen.

Sanity check

The official baseline feature set run through my booster settings gives macro-F1 0.539 vs the published 0.541, so the gains on the next slide come from the new features, not from a different setup.

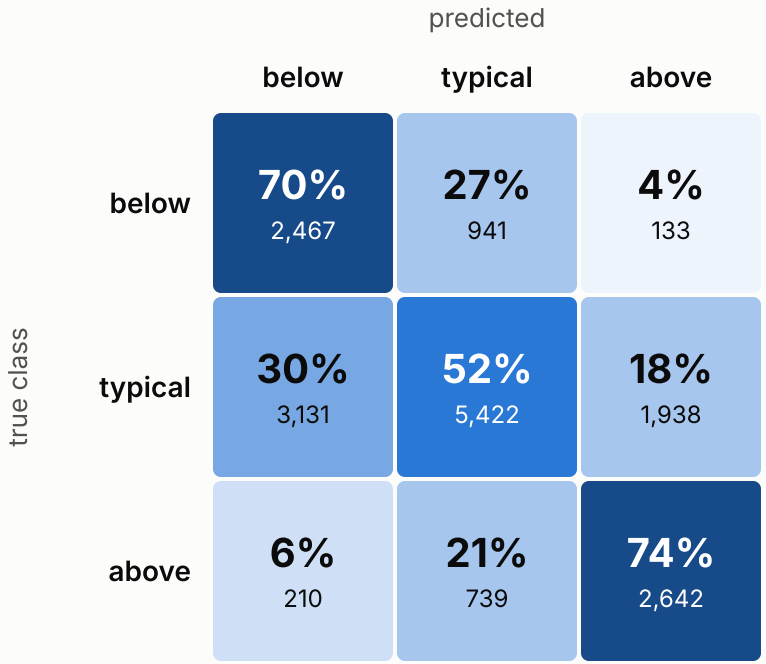
Both new blocks add signal: history +0.033, text +0.022



MODEL	F1	QWK	ACC
Always typical	0.249	0.000	0.595
Official baseline	0.541	0.458	0.547
Baseline feats, my GBM	0.539	0.460	0.544
+ author history	0.572	0.510	0.576
+ history + text	**0.594**	**0.540**	**0.598**

Scored with the course’s `evaluate.py` on all 17,623 test posts. Text alone reaches only 0.452, but it is complementary to history: stacked, it adds +0.022.

Extremes are found well; the cost is false alarms on typical posts



Row-normalised: share of each true class. Counts below.

CLASS	PRECISION	RECALL	F1
below	0.425	0.697	0.528
typical	0.763	0.517	0.616
above	0.561	0.736	0.636

- **Ordinal mistakes are rare:** only 343 posts (1.9%) are confused between *below* and *above*, which is why QWK rose with F1.
- **Recall on the extremes is high** (70% / 74%), but balanced weights push about half of the *typical* posts into an extreme class, so precision for *below* is only 0.43.
- **Trade-off:** accuracy (0.598) stays near the always-typical level, as expected when the objective is macro-F1.

What worked, what didn’t, and why

Worked

- **Recent momentum.** An author whose last posts beat their long-run median is likely to beat it again.
- **Stacked text probabilities** on top of metadata.
- **Balanced class weights** for a macro-F1 objective.

Weaker than expected

- **Text alone** (0.452). Wording says more about *who* writes than about how a post does against the author.
- **Follower count** is today’s audience, not the audience at posting time.

Caveats

- **Reactions are one snapshot** about 710 days after posting, so `age_days` is a collection artefact, not a property of the post.
- Hyperparameters were not tuned (time limit): one run, fixed seed.

Link to Tutorial 1-2: raw social text was *not* normalised (Q1). Char n-grams keep emoji, hashtags and placeholders as signals and are robust to informal spelling.

Where the remaining signal probably is

- **Multimodal fusion.** Image / PDF-cover embeddings (e.g. CLIP) and video poster frames, added as another stacked block (late fusion).
- **Sentence embeddings** instead of TF-IDF, plus a kNN “similar past posts of this author” feature (Tutorial Q3).
- **Decision thresholds tuned for macro-F1** on out-of-fold probabilities, instead of argmax.
- **LLM judge.** The dataset authors report macro-F1 0.627 for a zero-shot frontier model given the author’s last five posts (2,000-post sample). Its output could be a stacked feature.
- **Interpretability.** Permutation importance / SHAP to check that the model reads content, not collection artefacts (Tutorial Q5).

Ablation so far

0.539 metadata → **0.572** + author history (+0.033) → **0.594** + text (+0.022) → **?** + images / embeddings / LLM

Code, predictions and how to rerun

Submission

- Pull request **#14** to `Danil482/kairos-dataset`, branch `pasharodygin:lab1-rodygin`
- `solution.py`: features, text model, booster
- `REPORT.md`: written report
- `submissions/predictions_rodygin.csv`: 17,623 test predictions

Rerun

```
pip install -r requirements.txt
python solution.py          # ~7 min, with text
python solution.py --no-text   # ~15 s
python evaluate.py \
  --predictions results/predictions/solution.csv
```

Needs only `posts/` and `split.json`. Labels are recomputed if `labels.csv` is absent.

0.5935	0.5401	0.5976
macro-F1	QWK	accuracy

Final model on the test authors, verified with the course's `evaluate.py`.